

1. TITLE PAGE

LegalEase: AI-Powered Legal Document Generator

Sub-Title: Creating Dynamic Previews and Export Options

Naan Mudhalvan Project Report

Project Domain: Artificial Intelligence • Web Development • Document Automation

Academic Year: 2026–2027

A practical project for generating, previewing, and exporting structured legal document drafts.

2. ABSTRACT

LegalEase is an AI-powered web application designed to simplify the preparation of structured legal document drafts. The system accepts user information, processes it through the document-generation workflow, and presents the result in a readable format.

This project focuses on **dynamic previews and export options**. A dynamic preview allows users to see how their generated document will look before downloading or printing it. Export options provide a convenient way to save the final draft in a suitable format.

The feature improves usability by connecting document generation, review, formatting, and output into one continuous workflow.

3. INTRODUCTION

A document generator should not stop after producing text. Users need to inspect the result, identify missing information, and confirm formatting before saving or sharing it. LegalEase therefore includes a preview stage between generation and export.

The preview screen can update whenever the user changes relevant information or requests a new generation. The export layer can convert the reviewed content into a downloadable file while preserving headings, paragraphs, spacing, and basic document structure.

The overall objective is to make the application practical, understandable, and efficient for users.

4. PROBLEM STATEMENT

Traditional document-generation workflows may require users to generate a file, open it in another application, identify mistakes, return to the website, and repeat the process. This creates unnecessary steps.

Problem: Develop a system that provides an immediate dynamic preview and convenient export choices after document generation.

The solution should support readable previews, refresh or regenerate actions, print-friendly layouts, downloadable output, error handling, and consistent formatting across the preview and exported document.

5. OBJECTIVES

Primary objective: To design and develop dynamic preview and export functionality for the LegalEase application.

Specific objectives:

- Display generated content immediately after processing.
- Update the preview when document data changes.
- Provide clear review and editing controls.
- Maintain consistent formatting between preview and export.
- Support suitable downloadable formats such as PDF or DOCX where implemented.
- Provide print-friendly output.
- Handle export failures with clear user feedback.
- Keep the workflow simple for first-time users.

6. SYSTEM WORKFLOW

The complete workflow can be represented as: User Input → Validation → AI/Template Generation → Dynamic Preview → Review/Edit → Export Preparation → Download/Print.

The preview stage acts as a quality checkpoint. Users can review the document before creating a final file.

A modular workflow also makes it easier to add future export formats without changing the core document-generation logic.

7. DYNAMIC PREVIEW CONCEPT

A dynamic preview is a live representation of the document generated from current application data. Instead of showing only a plain text response, the interface can render headings, sections, paragraphs, lists, dates, and other supported document elements in a document-like layout.

When the underlying data changes, the preview can be refreshed or updated. This gives users immediate visual feedback and helps them understand the final structure.

The preview should remain readable and should not expose internal prompts, API responses, or technical processing details.

8. PREVIEW PAGE DESIGN

The preview page should have a clear document area surrounded by useful controls. A suitable layout may contain a top toolbar, document canvas, page information, and action buttons.

Suggested controls include **Edit**, **Regenerate**, **Preview**, **Download PDF**, **Download DOCX**, and **Print**, depending on the implemented features.

The interface should visually separate the document from controls so that users can concentrate on reviewing the content.

9. LIVE DATA BINDING

Dynamic previews depend on a reliable connection between form data and the preview component. When a user changes a supported field, the application can update the corresponding document section.

For simple applications, the preview may refresh after form submission. For more interactive applications, JavaScript can update selected elements without reloading the entire page.

Data binding should be predictable. User input must be escaped and validated so that unexpected markup or unsafe content is not inserted into the page.

10. EDIT AND REGENERATE FLOW

A useful preview should allow users to identify a problem and return to the relevant information rather than starting again from the beginning.

The **Edit** action can return the user to the input form with previous values preserved. After changes are submitted, the system can regenerate or re-render the preview.

The **Regenerate** action can request a new AI-assisted draft when the application supports it. The interface should communicate that regeneration may produce different wording and that users should review the new result.

11. DOCUMENT FORMATTING

A professional preview requires consistent typography and spacing. Headings should be visually distinct, paragraphs should have comfortable line spacing, and page margins should resemble the selected export format.

Formatting rules can include document title, section headings, numbered clauses, dates, signatures, and footer information where supported.

The same logical document structure should be used by both the preview renderer and export generator so that users do not receive an unexpectedly different document after downloading.

12. PDF EXPORT

PDF is useful when users want a fixed-layout document for printing, submission, or sharing. The export process converts the reviewed document into a PDF while attempting to preserve page structure and formatting.

A PDF export workflow may include: collect current document data, validate it, render the document structure, create the PDF, save it temporarily or stream it to the browser, and return a download response.

The application should handle PDF-generation errors gracefully and avoid exposing internal file paths or server errors to the user.

13. DOCX AND OTHER EXPORT OPTIONS

DOCX can be useful when users need to continue editing a generated draft in a word processor. Depending on the project implementation, LegalEase can also provide plain-text or print options.

Each export format has different layout capabilities. The system should therefore use a shared document model rather than assuming that HTML formatting will map perfectly to every format.

The interface should clearly label each option so users understand what they are selecting. Unsupported options should not be displayed as if they are available.

14. PRINT-FRIENDLY DESIGN

Printing requires a different presentation from normal web browsing. Navigation bars, large buttons, and interactive controls can be hidden from the print version using print-specific CSS.

The printable document should use suitable margins, readable text, page breaks, and consistent headers or footers where required.

A print preview action can help users verify the physical layout before printing. This complements PDF export and gives users another practical output path.

15. FRONTEND TECHNOLOGIES

HTML: Structures the preview area, controls, document sections, and export actions.

CSS: Defines page-like layouts, typography, spacing, responsive behavior, and print styles.

JavaScript: Supports dynamic updates, preview refresh, buttons, loading states, and client-side interactions.

Jinja/Flask templates: Can render server-generated document content safely into the preview page.

The exact technologies can be adjusted to match the student's implementation, but the architecture should keep presentation separate from generation logic.

16. BACKEND & EXPORT LOGIC

The backend receives the user's validated data and prepares a structured document representation. The preview route can return an HTML representation, while an export route can generate the requested file.

A conceptual route flow is: receive request → validate document state → load structured content → select output format → generate file → return download response.

Export functions should be separated from route definitions where practical. This improves maintainability and makes it easier to add new formats later.

17. TESTING

Testing should verify both the visual preview and the actual exported file.

Functional tests: valid generation, preview display, edit flow, regenerate flow, PDF export, DOCX export where available, and print action.

Validation tests: missing fields, invalid values, empty generated content, and unsupported formats.

Visual tests: headings, spacing, page breaks, long paragraphs, tables or lists where supported, and mobile responsiveness.

Error tests: export service failure, invalid file generation, timeout, and unexpected server errors.

18. SECURITY & RESPONSIBLE USE

Legal documents may contain personal or confidential information. Export functionality should therefore be designed with privacy and security in mind.

Temporary generated files should be handled securely and removed when they are no longer needed where appropriate. File names should avoid exposing sensitive information. User input should be validated and safely rendered to prevent injection vulnerabilities.

The application should clearly state that AI-generated legal documents are drafts and may contain errors. Users should seek qualified professional review when the document has legal significance.

19. FUTURE ENHANCEMENTS

Future versions can add multiple export templates, customizable page styles, digital signatures where legally and technically appropriate, document version history, multilingual Tamil/English output, watermark controls, saved drafts, and cloud storage with appropriate security.

A side-by-side editor and preview can make editing faster. Automatic page numbering, headers, footers, clause numbering, and configurable templates can improve professional formatting.

Export analytics and user feedback can also help identify common problems and guide future improvements.

20. CONCLUSION & VIVA SUMMARY

LegalEase demonstrates how AI-assisted generation can be combined with dynamic previews and export functionality to create a complete document workflow. The preview provides a quality-check stage, while export options transform the reviewed draft into a practical output.

Project Title: LegalEase: AI-Powered Legal Document Generator

Sub-title: Creating Dynamic Previews and Export Options

Core feature: Generate → Preview → Review/Edit → Export/Print.

Key technologies: HTML, CSS, JavaScript, Python, Flask/Jinja, AI/document-generation services, and PDF/DOCX generation libraries where configured.

Key benefit: Users can inspect and obtain their generated document without leaving the application workflow.

Final statement: LegalEase is a strong Naan Mudhalvan project concept demonstrating AI, web development, document automation, user experience, and practical software engineering in one application.